

TenantSea Backend API Guide

This document explains what each API endpoint does, who can call it, and includes copy/paste examples.

Base URL, versioning, and docs

- **Base URL:** `/api/v1`
- **Swagger UI:** `/api/docs`
- **OpenAPI JSON:** `/api/docs-json`

Authentication

- Most endpoints require a JWT in: `Authorization: Bearer <accessToken>`
- Login returns `accessToken` .

Multi-tenancy (`tenantId`)

- The system supports multiple organizations ("tenants") in one database.
- Most requests include `tenantId` (in JSON body for POST or query string for GET).
- The backend compares request `tenantId` to the JWT `tenantId` and rejects mismatches.

Health

GET `/health`

Checks the API and DB connectivity.

Example:

```
curl -s http://localhost:3100/api/v1/health
```

Response:

```
{ "status": "ok", "db": "up" }
```

Auth

POST `/auth/register`

Creates a tenant (if needed) + user account and sends an email verification link.

Body fields:

- `tenantId` , `name` , `email` , `password` , `role` , optional `phone`

Example:

```
curl -s -X POST http://localhost:3100/api/v1/auth/register \  
-H "Content-Type: application/json" \  
-d '{  
  "tenantId": "acme-corp",  
  "name": "Jane Doe",  
  "email": "jane@example.com",
```

```
"password": "SecurePass123!",  
"role": "tenant"  
'}
```

POST /auth/register-landlord

Same as register, but forces the role to `landlord`.

POST /auth/register-agent

Same as register, but forces the role to `letting_agent`.

POST /auth/verify-email

Verifies a user's email using the token from the verification email.

Example:

```
curl -s -X POST http://localhost:3100/api/v1/auth/verify-email \  
-H "Content-Type: application/json" \  
-d '{"token": "verification_token_here", "tenantId": "acme-corp"}'
```

POST /auth/resend-verification

Re-sends the verification email.

Example:

```
curl -s -X POST http://localhost:3100/api/v1/auth/resend-verification \  
-H "Content-Type: application/json" \  
-d '{"email": "jane@example.com", "tenantId": "acme-corp"}'
```

POST /auth/login

Logs in using email/password (optionally include `tenantId` if the same email can exist in multiple tenants).

Example:

```
curl -s -X POST http://localhost:3100/api/v1/auth/login \  
-H "Content-Type: application/json" \  
-d '{"email": "jane@example.com", "password": "SecurePass123!", "tenantId":  
"acme-corp"}'
```

Response contains:

- `accessToken` (JWT)
- `user`

GET /auth/me (JWT required)

Returns your profile from the token (safe fields only).

Example:

```
curl -s http://localhost:3100/api/v1/auth/me \
-H "Authorization: Bearer <accessToken>"
```

POST /auth/forgot-password

Issues a reset token (stored in DB) and emails a reset link. Response is always generic.

Example:

```
curl -s -X POST http://localhost:3100/api/v1/auth/forgot-password \
-H "Content-Type: application/json" \
-d '{ "email": "jane@example.com", "tenantId": "acme-corp" }'
```

POST /auth/reset-password

Resets password using the reset token.

Example:

```
curl -s -X POST http://localhost:3100/api/v1/auth/reset-password \
-H "Content-Type: application/json" \
-d '{ "token": "abc123resettoken", "password": "NewSecurePass123!", "tenantId": "acme-corp" }'
```

Users (JWT + tenant-scoped)

All endpoints in this section require:

- Authorization: Bearer <accessToken>
- tenantId query/body matching your JWT tenant

POST /users (Roles: ADMIN , LANDLORD , LETTING_AGENT)

Creates a user in the tenant (password is hashed).

Example:

```
curl -s -X POST http://localhost:3100/api/v1/users \
-H "Authorization: Bearer <accessToken>" \
-H "Content-Type: application/json" \
-d '{
  "tenantId": "acme-corp",
  "name": "New Tenant",
  "email": "newtenant@example.com",
  "password": "SecurePass123!",
  "role": "tenant"
}'
```

GET /users?tenantId=...&page=...&limit=...&search=...

Lists users.

Example:

```
curl -s "http://localhost:3100/api/v1/users?tenantId=acme-corp&page=1&limit=20" \
-H "Authorization: Bearer <accessToken>"
```

GET `/users/:id?tenantId=...`

Gets a user by id.

Properties (JWT + tenant-scoped)

POST `/properties` (Roles: `LANDLORD`, `LETTING_AGENT`, `ADMIN`)

Creates a property record for the tenant.

Example:

```
curl -s -X POST http://localhost:3100/api/v1/properties \
-H "Authorization: Bearer <accessToken>" \
-H "Content-Type: application/json" \
-d '{
  "tenantId": "acme-corp",
  "title": "2-bed flat in Camden",
  "address": "12 High Street, London",
  "landlordId": "<userId>",
  "bedrooms": 2,
  "rentAmount": 1500,
  "currency": "GBP"
}'
```

GET `/properties?tenantId=...&page=...&limit=...&search=...`

Lists properties.

Tenancies (JWT + tenant-scoped)

POST `/tenancies` (Roles: `LANDLORD`, `LETTING_AGENT`, `ADMIN`)

Creates a tenancy (linking tenant user, landlord, and property).

Example:

```
curl -s -X POST http://localhost:3100/api/v1/tenancies \
-H "Authorization: Bearer <accessToken>" \
-H "Content-Type: application/json" \
-d '{
  "tenantId": "acme-corp",
  "propertyId": "<propertyId>",
  "tenantUserId": "<tenantUserId>",
  "landlordId": "<landlordId>",
  "rentAmount": 1500,
```

```
"currency": "GBP",
"startDate": "2026-01-01",
"status": "active"
}'
```

GET `/tenancies?tenantId=...&page=...&limit=...&search=...`

Lists tenancies.

Payments (JWT + tenant-scoped)

POST `/payments/initiate`

Creates a payment record (does not charge a provider; it records intent + metadata).

Example:

```
curl -s -X POST http://localhost:3100/api/v1/payments/initiate \
-H "Authorization: Bearer <accessToken>" \
-H "Content-Type: application/json" \
-d '{
  "tenantId": "acme-corp",
  "tenancyId": "<tenancyId>",
  "payerId": "<payerUserId>",
  "amount": 1500,
  "currency": "GBP",
  "provider": "flutterwave",
  "reference": "pay_ref_123",
  "dueDate": "2026-02-01",
  "status": "pending"
}'
```

POST `/payments/verify` (Roles: **LANDLORD** , **ADMIN**)

Manually verifies/updates a payment's status and verified amount.

Example:

```
curl -s -X POST http://localhost:3100/api/v1/payments/verify \
-H "Authorization: Bearer <accessToken>" \
-H "Content-Type: application/json" \
-d '{
  "tenantId": "acme-corp",
  "paymentId": "<paymentId>",
  "status": "paid",
  "verifiedAmount": 1500
}'
```

POST `/payments/schedules`

Creates a recurring payment schedule for a tenancy (and creates a notification).

Example:

```
curl -s -X POST http://localhost:3100/api/v1/payments/schedules \
-H "Authorization: Bearer <accessToken>" \
-H "Content-Type: application/json" \
-d '{
  "tenantId": "acme-corp",
  "tenancyId": "<tenancyId>",
  "amount": 1500,
  "currency": "GBP",
  "frequency": "monthly",
  "nextDueDate": "2026-02-01"
}'
```

GET `/payments/schedules?tenantId=...&tenancyId=...`

Lists schedules.

POST `/payments/reconcile?tenantId=...` (Roles: `LANDLORD` , `ADMIN`)

Checks for overdue schedules and creates notifications for those missing a PAID payment at the due date.

GET `/payments/history?tenantId=...&tenancyId=...&page=...&limit=...`

Lists payments.

Trust (JWT + tenant-scoped)

GET `/trust/dashboard?tenantId=...` (Roles: `LANDLORD` , `ADMIN` , `LETTING_AGENT`)

Returns a landlord-style dashboard summary based on payments and tenancies.

GET `/trust/:tenantUserId?tenantId=...`

Returns a trust profile for a tenant user:

- Privileged roles can view any profile in the tenant.
 - Tenants can view their own profile only.
-

Audit (JWT + tenant-scoped)

GET `/audit?tenantId=...` (Role: `ADMIN`)

Lists the latest audit logs (currently returns the last 100 records).

Notifications (JWT + tenant-scoped)

GET `/notifications?tenantId=...&userId=...&read=true|false&page=...`

Lists notifications. Non-privileged users only see their own notifications.

POST `/notifications/:id/read`

Marks a notification read/unread.

Example:

```
curl -s -X POST http://localhost:3100/api/v1/notifications/<id>/read \
-H "Authorization: Bearer <accessToken>" \
-H "Content-Type: application/json" \
-d '{ "tenantId": "acme-corp", "read": true }'
```

Realtime (Socket.IO)

The server emits two main event channels:

- `notification` (payload includes `event: notification:created|notification:updated`)
- `payment:update` (payload includes `event: payment:updated`)